Ch03 错误和异常处理

- 本章要点：
  - ✓3.1 程序的错误
  - ✓3.2 异常处理
  - ✓3.3程序的基本调试方法

程序的错误

- 语法错误、运行时错误和逻辑错误
  - 语法错误是指其源代码中拼写语法错误，这些错误导致Python编译器无法把Python源代码转换为字节码，故也称之为编译错误
- 【例3.1】Python语法错误示例（syntax_error.py）

```
print("Good Luck!"
print("你今天的幸运随机数是：", random.choice(range(10)))
```

```
1 print("Good Luck!"
2 print("你今天的幸运随机数是：", random.choice(range(10)))
3

  Cell In[6], line 1
    print("Good Luck!"

SyntaxError: '(' was never closed
```

运行时错误

- 运行时错误是在
  - 程序中没有导入相关的模块（例如，**import random**）时，解释器将在运行时抛出**NameError**错误信息
  - 程序中包括零除运算，解释器将在运行时抛出**ZeroDivisionError**错误信息
  - 程序中试图打开不存在的文件，解释器将在运行时抛出**FileNotFoundError**错误信息

• 【例3.2】Python运行时错误（没有导入相关的模块）

```
print("Good Luck!")
print("你今天的幸运随机数是：", random.choice(range(10)))
```

```
: 1 print("Good Luck!")
  2 print("你今天的幸运随机数是：", random.choice(range(10)))
  3

Good Luck!

---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[7], line 2
      1 print("Good Luck!")
----> 2 print("你今天的幸运随机数是：", random.choice(range(10)))

NameError: name 'random' is not defined
```

【例3.3】Python运行时错误（零除错误）

```
a=1
b=0
c=a/b
```

```
1 a=1
2 b=0
3 c=a/b
4
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[8], line 3
      1 a=1
      2 b=0
----> 3 c=a/b

ZeroDivisionError: division by zero
```

逻辑错误

- 逻辑错误是程序可以执行（程序运行本身不报错），但执行结果不正确。对于逻辑错误，Python解释器无能为力，需要根据结果来调试判断
- 【例3.4】Python逻辑错误示例

```
import math
a=1; b=2; c=1
x1 = -b + math.sqrt(b*b-4*a*c)/2*a  #公式有误，故结果不正确
x2 = -b - math.sqrt(b*b-4*a*c)/2*a  #公式有误，故结果不正确
print(x1, x2)                       #输出：-2.0 -2.0
```

- 程序计算一元二次方程$ax^2 + bx + c = 0$的两个根：$x = \frac{-b \pm \sqrt{b^2-4ac}}{2a}$。方程$x^2 + 2x + 1 = 0$正确的解为x1=x2=-1。但由于计算公式有误（正确公式为(-b + math.sqrt(b*b-4*a*c))/(2*a)以及(-b - math.sqrt(b*b-4*a*c))/(2*a)），故结果不正确

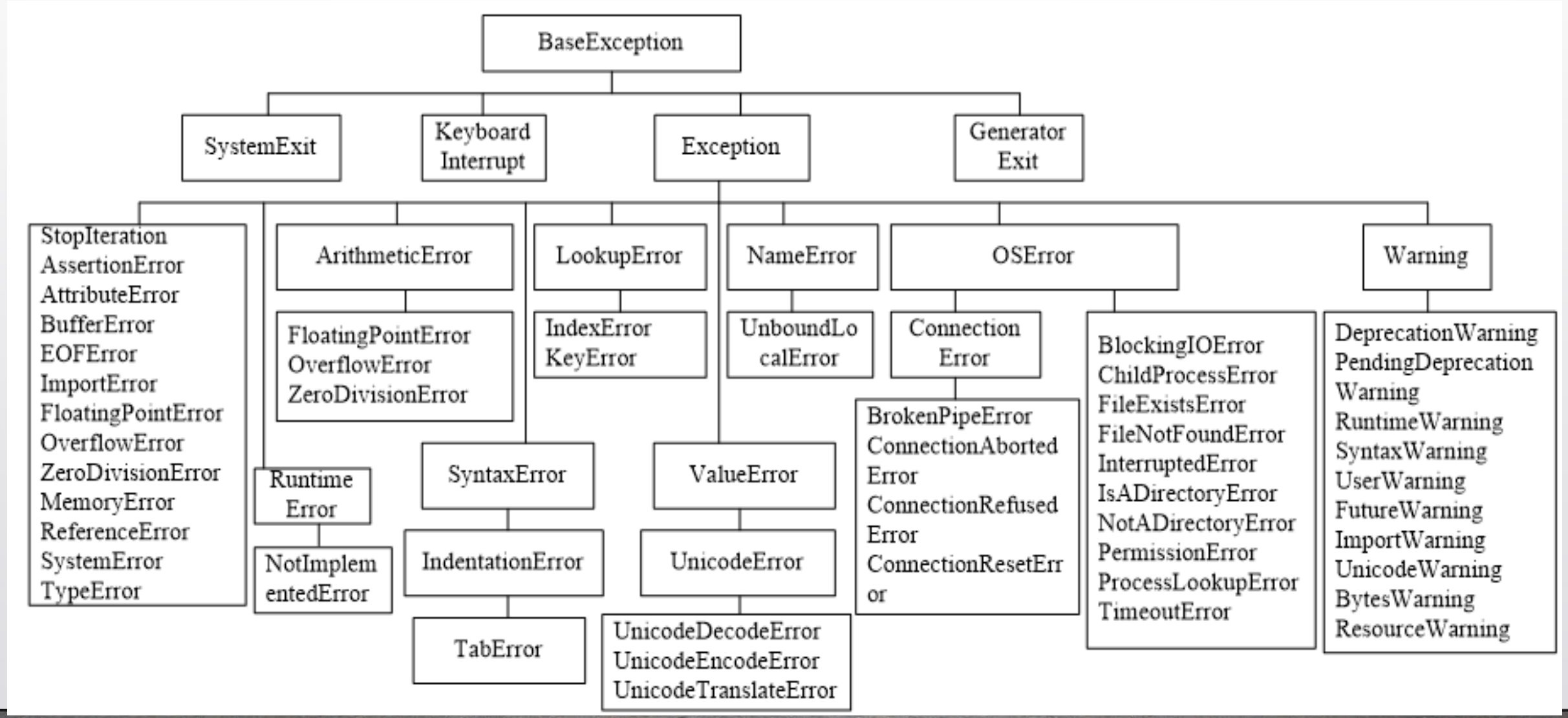
异常处理

在程序运行过程中，如果产生错误，则抛出异常，通过try语句来定义代码块，以运行可能抛出异常的代码；通过except语句，可以捕获特定的异常并执行相应的处理；try语句块执行结束，且没有发生异常，else语句块执行，通过finally语句，可以保证即使产生异常（处理失败），也可以在事后清理资源等

```
def read_file():
    try:
        打开文件          #可能产生错误：文件不存在
        读取文件内容      #可能产生错误：无读取权限
        关闭文件
    except FileNotFoundError: #捕获异常：无法打开文件
        #异常处理逻辑
    except PermissionError:    #捕获异常：无读取权限
        #异常处理逻辑
```

异常处理机制可以把错误处理和正常代码逻辑分开，从而可以更加高效地实现错误处理，增加程序的可维护性。

内置的异常类


BaseException
SystemExit
Keyboard Interrupt
Exception
Generator Exit
StopIteration
AssertionError
AttributeError
BufferError
EOFError
ImportError
FloatingPointError
OverflowError
ZeroDivisionError
MemoryError
ReferenceError
SystemError
TypeError
ArithmeticError
FloatingPointError
OverflowError
ZeroDivisionError
LookupError
IndexError
KeyError
NameError
UnboundLocalError
OSError
Connection Error
BrokenPipeError
ConnectionAborted Error
ConnectionRefused Error
ConnectionResetError
BlockingIOError
ChildProcessError
FileExistsError
FileNotFoundError
InterruptedError
IsADirectoryError
NotADirectoryError
PermissionError
ProcessLookupError
TimeoutError
Warning
DeprecationWarning
PendingDeprecation Warning
RuntimeWarning
SyntaxWarning
UserWarning
FutureWarning
ImportWarning
UnicodeWarning
BytesWarning
ResourceWarning
Runtime Error
NotImplemented Error
SyntaxError
IndentationError
TabError
ValueError
UnicodeError
UnicodeDecodeError
UnicodeEncodeError
UnicodeTranslateError

【例3.5】常见异常示例（1）

（1）NameError。尝试访问一个未申明的变量。

```
>>> noname   #报错。NameError: name 'noname' is not defined
```

（2）SyntaxError。语法错误。

```
>>> int a      #报错。SyntaxError: invalid syntax
```

（3）AttributeError。访问未知对象属性。

```
>>> a=1
>>> a.show()  #报错。AttributeError: 'int' object has no attribute 'show'
```

（4）TypeError。类型错误。

```
>>> 11 + 'abc'  #报错。TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

（5）ValueError。数值错误。

```
>>> int('abc')  #报错。ValueError: invalid literal for int() with base 10: 'abc'
```

语法错误（SyntaxError） 若在代码静态解析阶段出现，无法被try捕获；动态执行时可捕获。其余异常均为运行时错误，均可通过try...except捕获。

【例3.5】常见异常示例（2）

（6）ZeroDivisionError。零除错误。

```
>>> 1/0      #报错。ZeroDivisionError: division by zero
```

（7）IndexError。索引超出范围。

```
>>> a=[10,11,12]
>>> a[3]     #报错。IndexError: list index out of range
```

（8）KeyError。字典关键字不存在。

```
>>> m = {'1':'yes', '2':'no'}
 >>> m['3']     #报错。KeyError: '3'
```

Python虚拟机捕获处理异常

大部分由程序错误而产生的错误和异常，一般由Python虚拟机自动抛出

- 【例3.6】Python虚拟机捕获处理异常示例（pvmexcept.py）

```
i1 = 1
i2 = 0
print(i1/i2)
```

程序运行后将显示“ZeroDivisionError: division by zero”报错信息

使用try…except…else…finally语句捕获处理异常

```
try:
    可能产生异常的语句
except Exception1:                #捕获异常 Exception1
    发生异常时执行的语句
except (Exception2, Exception3):  #捕获异常 Exception2、Exception3
    发生异常时执行的语句
except Exception4 as e:           #捕获异常 Exception4，其实例为 e
    发生异常时执行的语句
except:                           #捕获其它所有异常
    发生异常时执行的语句
else:                             #无异常
    无异常时执行的语句
finally:                          #不管发生异常与否，保证执行
    不管发生异常与否，保证执行的语句
```

程序的基本调试方法

```
>>> if 1 > 2     #报错。SyntaxError: invalid syntax
>>> Print('abc')  #报错。NameError: name 'Print' is not defined
>>> x           #报错。NameError: name 'x' is not defined
```

- 语法错误的调试
  - 根据Python解释器抛出的异常，分别判断产生异常的原因：
  - 第一条语句产生的SyntaxError表示语法错误（if复合语句后没有冒号）
  - 第二条语句产生的NameError表示名称错误（拼写错误，应该为print）
  - 第三条语句产生的NameError表示名称错误（未定义变量）

对于编译错误，Python解释器会直接抛出异常。用户可以根据输出的异常信息修改程序代码。

运行时错误的调试

- **【例3.7】运行时错误调试示例**

```
>>> f = open('abc.txt')  #文件或者目录不存在：FileNotFoundError
>>> a=1;b=0
>>> c=a/b        #零除溢出：ZeroDivisionError
>>> 123 + 'abc'     # TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

- 根据Python解释器运行时抛出的异常，分别判断产生异常的原因：
- 第一条语句产生的FileNotFoundError表示打开不存在的文件
- 第三条语句产生的ZeroDivisionError表示零除错误
- 第四条语句产生的TypeError表示不同类型对象值不能相加

对于运行时错误，Python解释器也会抛出异常，在代码中可以通过try...except语句捕获并处理。如果程序中没有使用try...except语句，则Python解释器将直接打印出异常信息。

逻辑错误的调试

- 断点跟踪、输出信息等方法。
- 集成开发环境（IDE）可以设置断点，并查看变量等。
- 通过print语句输出程序运行过程中变量值（跟踪信息），是观察和调试程序运行逻辑正确性的有效方法。

【例】倒计时小程序（countdown.py）

- 递归函数小程序。程序模拟倒计）

```
import time
def countdown(n):
    """从n倒计数到0"""
    if n <= 0: #基本情况
        print("时间到！！！")
    else: #递归步骤
        time.sleep(1) #睡眠1秒钟
        print(n) #输出倒数的数字
        countdown(n-1) #递归调用
if __name__ == "__main__":
    countdown(3)
```

程序正常运行结果如下所示

3

2

1

时间到!!!

【例】使用IDLE调试器调试运行程序示例程序countdown.py

- （1）运行IDLE并打开程序源代码countdown.py
- （2）设置断点。在想要调试的语句上，通过鼠标右键快捷菜单【Set Breakpoint】设置断点，设置了断点的代码行底色为黄色，也可以通过【Clear Breakpoint】清除已经设置的断点
- （3）打开调试器。在IDLE窗口，按快捷键【F5】，程序开始运行。在弹出的Python解释器命令行窗口，通过执行菜单命令【Debug】|【Debugger】，打开调试器窗口
- （4）调试运行程序。在IDLE窗口，按快捷键【F5】，程序开始运行。程序在代码的第一行中断暂停。底色为灰色的代码为当前执行代码行。用户可以通过【Debug Control】窗口查看程序运行状态（调用堆栈Stack、局部变量Locals、全局变量Globals），执行下一步调试命令
- （5）在【Debug Control】窗口中单击运行调试命令【Go】，使程序运行到下一个断点处中断暂停
- （6）查看程序运行状态。查看局部变量Locals和全局变量Globals。此时n为3
- （7）在【Debug Control】窗口中单击运行调试命令【Over】，单步执行命令。由于n为3，故程序跳转到else后的第一条语句
- （8）继续执行其它调试命令

